UNIVERSIDAD MARIANO GÁLVEZ

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

Manual Técnico — EDUVIAGT

Instalación, configuración y operación

Proyecto	EDUVIAGT — Sistema Integral de Gestión de Becas
Curso	0910-037 · Análisis de Sistemas II · 8.º Ciclo
Catedrático / PO	Ing. Mario Fuentes
Equipo	EDUVIAGT — 6 integrantes
Versión	1.0
Fecha	2026-10-29

Quiénes somos

EDUVIAGT es la empresa consultora seleccionada por el Ministerio de Educación de Guatemala para analizar, diseñar e implementar la Plataforma Nacional para la Gestión Integral de Becas. Somos un equipo de 6 integrantes con roles Scrum definidos, que aplica prácticas reales de ingeniería de software.

Cómo trabajamos

- **Metodología:** Scrum con sprints de 2 semanas (9 sprints, agosto 2026 – enero 2027).
- **Ceremonias:** Sprint Planning, Daily Standup (15 min), Sprint Review, Sprint Retrospective, Backlog Refinement.
- **Roles:** Scrum Master (Chiroy Vásquez, Luis David), Product Owner simulado, Development Team (5 integrantes).
- **Definition of Done:** código + pruebas + code review + documentación + despliegue verificado en staging.
- **Control de versiones:** Git con ramas `feature/*`, integración en `develop`, producción en `master` protegida, PR obligatorio.
- **Calidad:** TDD, ESLint + Prettier, Jest + Supertest, Playwright + axe, pipeline CI de 6 jobs.

Manual Técnico

Proyecto: *EDUVIAGT — Sistema Integral de Gestión de Becas* ***Curso:*** *0910-037 · Análisis de Sistemas II · 8.º Ciclo* ***Versión:*** *1.0 ·* ***Fecha:*** *2026-10-28* ***Audiencia:*** *desarrolladores, responsables de tecnologia y de operacion del proyecto; revisores externos*

Este manual documenta cómo **instalar, configurar, ejecutar, probar y desplegar** EDUVIAGT, y describe la estructura técnica del proyecto. Para la justificación de las decisiones de diseño, véanse 04-documento-arquitectura.md y 05-justificacion-arquitectura.md.

1. Resumen técnico

Aspecto	Detalle
Tipo de aplicación	Aplicación web (SPA con SSR parcial) con API REST
Monorepo	npm workspaces + Turborepo
Backend	NestJS 10 + TypeScript 5 + Prisma 5
Frontend	Next.js 14 (App Router) + React 18 + Tailwind CSS
Base de datos	PostgreSQL 16
Autenticación	Passport + JWT (access 15 min / refresh 7 d) + bcrypt
Autorización	Chain of Responsibility *fail-closed* (usuario → rol → denegar)
Almacenamiento de documentos	Sistema de archivos local detrás de la interfaz `DocumentStorage`
PDF	Puppeteer (Chromium headless) detrás de la interfaz `PdfRenderer`

Aspecto	Detalle
Asistente IA	FTS de PostgreSQL + LLM OpenAI-compatible opcional, detrás de `ProveedorIA`
Pruebas	Jest + Supertest (API), Playwright + axe (web)
CI/CD	GitHub Actions (6 jobs) → SSH → PM2 + nginx
Node.js	≥ 20 · **npm** 10

2. Estructura del repositorio

```
EDUVIAGT-v1.1/
■■■ apps/
■   ■■■ api/                      Backend NestJS
■   ■   ■■■ prisma/
■   ■   ■   ■■■ schema.prisma     36 entidades, 6 enums
■   ■   ■   ■■■ migrations/       Migraciones versionadas
■   ■   ■   ■■■ seed.ts           Seed idempotente
■   ■   ■   ■■■ semilla/          Datos de demostración (4 archivos + verificador)
■   ■   ■■■ src/
■   ■       ■■■ main.ts           Bootstrap, Swagger, pipes globales
■   ■       ■■■ app.module.ts     Composición raíz (17 módulos)
■   ■       ■■■ common/           Guards, decoradores, interceptores, filtros
■   ■       ■■■ auth/             Registro, login, refresh, perfil, eliminación
■   ■       ■■■ users/            Roles, permisos, excepciones, configuración
■   ■       ■■■ catalogos/        8 catálogos
■   ■       ■■■ convocatorias/    CRUD + máquina de estados + documentos + montos
■   ■       ■■■ solicitudes/      Solicitud, perfiles, documentos, montos, PDF
■   ■       ■■■ evaluaciones/     Asignación y puntajes
■   ■       ■■■ comites/          Comités y miembros
■   ■       ■■■ sesiones/         Sesiones, votos, quórum, decisiones
■   ■       ■■■ reportes/         Agregaciones + export CSV
■   ■       ■■■ asistente/        Proxy IA + providers + base de conocimiento
■   ■       ■■■ audit/            Auditoría
■   ■       ■■■ bitacora/         Bitácora append-only
■   ■       ■■■ notificaciones/   Notificaciones + SSE
■   ■       ■■■ storage/          Adapter de almacenamiento
■   ■       ■■■ prisma/           PrismaService
■   ■■■ web/                      Frontend Next.js
■       ■■■ e2e/                  Suites Playwright (+ axe)
■       ■■■ src/
■           ■■■ app/              25 rutas (App Router)
■           ■■■ components/       Componentes reutilizables
■           ■■■ context/          Contextos de sesión
■           ■■■ hooks/            Hooks de dominio
■           ■■■ lib/              Cliente de la API y tipos
■           ■■■ styles/           Design system
■■■ docs/                         Documentación (ver docs/README.md)
■   ■■■ entregables/              Entregables formales del curso
■   ■■■ architecture/             Arquitectura, ADR, diagramas, SOLID, patrones
■   ■■■ scrum/                    Evidencias de las ceremonias por sprint
■   ■■■ standards/                Convenciones de código, Git, testing, seguridad
■   ■■■ operations/               Setup, despliegue, troubleshooting
■   ■■■ manual-usuario/           Manuales por rol (6)
■   ■■■ pruebas/                  Baseline, caja blanca, funcional, a11y
■   ■■■ marca/                    Logos y tipografías
■   ■■■ team/                     Miembros y roles
■■■ scripts/                      dev-up.ps1, deploy-pm2.sh
■■■ storage/                      Documentos subidos (adapter FS)
■■■ turbo.json                    Pipeline de Turborepo
■■■ package.json                  Workspaces + scripts raíz
■■■ docker-compose.yml            PostgreSQL para desarrollo
```

3. Requisitos previos

Requisito	Versión	Nota
Node.js	≥ 20	`engines.node` del `package.json`
npm	10	`packageManager` del `package.json`
Docker Desktop	Actual	Solo para PostgreSQL en desarrollo
PowerShell 7 (`pwsh`)	Actual	Necesario para `npm run up`
Git	Actual	Control de versiones
Chromium (opcional)	Actual	Solo si se usa el render de PDF y no hay Chromium de Puppeteer

4. Instalación

4.1 Clonar e instalar dependencias

```
git clone <url-del-repositorio> EDUVIAGT-v1.1
cd EDUVIAGT-v1.1
npm ci
```

4.2 Configurar variables de entorno

```
cp .env.example .env
# o en PowerShell:
Copy-Item .env.example .env
```

Variables requeridas:

Variable	Descripción	Valor de ejemplo
`DATABASE_URL`	Cadena de conexión a PostgreSQL	`postgresql://sigeb:sigeb123@localhost:5432/sigeb`
`JWT_SECRET`	Secreto de firma de los access tokens (≥ 32 caracteres)	`cambiar-por-un-secreto-largo`
`JWT_REFRESH_SECRET`	Secreto **distinto** de los refresh tokens	`cambiar-por-otro-secreto-largo`
`JWT_REFRESH_EXPIRATION`	Vigencia del refresh token	`7d`
`PORT`	Puerto de la API	`3000`
`CORS_ORIGIN`	Origen permitido del frontend	`http://localhost:3001`
`STORAGE_PATH`	Ruta raíz de almacenamiento de documentos	`./storage`
`AI_API_KEY`	**Opcional.** Si está vacío, el asistente usa la base de conocimiento	*(vacío)*
`AI_BASE_URL`	Endpoint compatible con OpenAI	`https://api.openai.com/v1`

Variable	Descripción	Valor de ejemplo
`CHROME_EXECUTABLE_PATH`	**Opcional.** Ruta a un Chromium del sistema para el PDF	*(vacío)*

Reglas de seguridad - `JWT_SECRET` *y* `JWT_REFRESH_SECRET` ***deben ser distintos****. El sistema verifica esta condición. -* ***Nunca*** *se versiona un* `.env` *real.* `.gitignore` *lo bloquea y el CI escanea secretos. - En staging y producción los valores reales se inyectan como secretos del entorno, no en el repositorio.*

4.3 Generar el cliente de Prisma y aplicar migraciones

```
cd apps/api
npx prisma generate
npx prisma migrate deploy
npx prisma db seed
```

El seed es **idempotente** y genera los catálogos, roles, permisos, la base de conocimiento del asistente (33 documentos), los usuarios demo y el conjunto completo de datos de demostración (51 convocatorias, 1,000 solicitudes, 250 postulantes, 30 evaluadores, 50 miembros de comité, 4 ciclos).

4.4 Ejecutar en desarrollo

```
# Opción recomendada — levanta PostgreSQL + API + web, aplica migraciones y seed
npm run up

# Detener todo
npm run up:stop

# Opción manual
docker compose up -d postgres
npm run dev            # turbo run dev (API en watch + web)
npm run dev:api        # solo API
npm run dev:web        # solo web
```

4.5 Accesos

Servicio	URL
Web	http://localhost:3001
API	http://localhost:3000/api
Swagger (OpenAPI)	http://localhost:3000/api/docs

4.6 Usuarios de demostración

Contraseña común: `Admin123!`

Rol	Correo
Administrador	`admin@EDUVIAGT.gov.gt`
Postulante	`postulante@demo.gt`
Evaluador	`evaluador@demo.gt`
Coordinador de comité	`coordinador@demo.gt`
Miembro de comité	`miembro@demo.gt`
Jefe de becas	`jefe@demo.gt`

5. Base de datos

5.1 Migraciones

```
# Crear una migración (desarrollo)
cd apps/api
npx prisma migrate dev --name <descripcion>

# Aplicar migraciones existentes (staging/producción)
npx prisma migrate deploy

# Ver el estado
npx prisma migrate status

# Abrir Prisma Studio
npx prisma studio
```

Reglas

- Una migración aplicada **no se edita**: se crea una migración correctiva.
- Toda migración debe ser **idempotente** respecto a los datos de demostración.
- El despliegue ejecuta `prisma migrate deploy` **antes** del build.

5.2 Modelo de datos

36 entidades agrupadas en 8 dominios. El detalle completo está en [`docs/architecture/database-schema.md`](../architecture/database-schema.md).

Dominio	Entidades
Autenticación y seguridad	Usuario, Rol, Permiso, RolPermiso, UsuarioPermiso, ConfiguracionUsuario
Catálogos	Genero, NivelAcademico, Departamento, Municipio, InstitucionEducativa, DocumentoTipo, Beca, CriterioEvaluacion
Convocatorias	Convocatoria, ConvocatoriaDocumentoRequerido, ConvocatoriaReglaMonto
Solicitudes	Solicitud, SolicitudPerfilAcademico, SolicitudPerfilFinanciero, SolicitudDocumento, SolicitudRespuesta, SolicitudAprobacion, HistorialEstado
Evaluación	Evaluacion, EvaluacionCriterio
Comités y sesiones	Comite, ComiteMiembro, Sesion, SesionAgenda, Voto, Decision
Auditoría y bitácora	AuditLog, BitacoraSistema, Notificacion
Asistente	AsistenteBaseConocimiento, AsistenteConversacion, AsistenteMensaje

Enums: `ConvocatoriaEstado`, `SolicitudEstado`, `SesionEstado`, `DecisionResultado`, `DocumentoEstado`, `TipoReglaMonto`.

5.3 Datos de demostración

```
prisma db seed
  ■■■ seed.ts:1432-1433  → generarEscenarios()
        ■■■ datos.ts      → catálogo canónico + 250 postulantes + 30 evaluadores + 50 miembros
        ■■■ ciclos.ts     → 4 ciclos de ~90 días
        ■■■ escenarios.ts → 1,000 solicitudes + evaluaciones + comités
        ■■■ verificar.ts  → aserciones de conteo (exit 0/1)
```

El verificador falla con código de salida 1 si los conteos no coinciden, lo que permite usarlo como control en el pipeline.

6. Ejecución de pruebas

```
# Suite completa del monorepo
npm test

# API
npm run test:api                 # unitarias
cd apps/api && npm run test:cov  # con cobertura y umbral

# Web (E2E)
cd apps/web && npm run test:e2e
```

Suite	Ubicación	Herramienta
Unitarias y de integración de la API	`apps/api/src/*/.spec.ts` (26 suites)	Jest + Supertest
E2E del frontend	`apps/web/e2e/` (8 suites)	Playwright + axe

Umbral de cobertura: statements 49 % · branches 40 % · functions 48 % · lines 49 %. El umbral está configurado en `apps/api/jest.config` y el pipeline falla si no se alcanza.

Detalle de la estrategia en [`docs/standards/testing-strategy.md`](../standards/testing-strategy.md) y en el [Plan de Pruebas Ágil](09-plan-pruebas-agil.md).

7. Pipeline de integración continua

Job	Qué hace	Condición
`api-tests`	lint + unitarias + `tsc` (api y web) + build de la API + smoke de arranque	Cada PR
`e2e-web`	Playwright + axe, con resiembra de `sigeb_test`	Cada PR
`docker-build`	Construye las imágenes Docker de API y web	`develop`, `master`
`deploy-staging`	SSH a la VM, `git fetch`, `npm ci`, `prisma migrate deploy`, build, `pm2 startOrRestart`	`develop`
`deploy-production`	Despliegue a producción	`master` + `PRODUCTION_READY=true` (deshabilitado en el semestre)
`lint-and-test`	Linter sobre el monorepo	Cada PR

Regla: un PR no se fusiona a `develop` con el pipeline en rojo.

8. Despliegue

8.1 Vía vigente — PM2 en una VM

```
# En la VM
cd /home/davi/sigeb
git fetch origin
git checkout develop
git reset --hard origin/develop
npm ci
npx prisma migrate deploy --schema apps/api/prisma/schema.prisma
npx prisma generate --schema apps/api/prisma/schema.prisma
npm run build
pm2 startOrRestart sigeb-api
pm2 startOrRestart sigeb-web
pm2 save
```

Configuración de PM2: `sigeb.pm2.config.cjs` con los procesos `sigeb-api` y `sigeb-web`. Proxy inverso: **nginx** con terminación TLS y reenvío de `/api/*` al proceso de la API.

8.2 Alternativa — Docker Compose

```
docker compose -f docker-compose.prod.yml up -d --build
```

Servicio	Imagen	Volumen
`postgres`	`postgres:16-alpine`	`postgres_data`
`api`	Dockerfile de `apps/api`	`storage_data` → `/data/storage`
`web`	Dockerfile de `apps/web`	—

Healthcheck del contenedor `api`: `GET /api/catalogos/generos`.

8.3 Checklist de despliegue

- [ ] Migraciones aplicadas (`prisma migrate deploy`).
- [ ] `JWT_SECRET` y `JWT_REFRESH_SECRET` distintos y fuera del repositorio.
- [ ] `CORS_ORIGIN` apunta al dominio real.
- [ ] `STORAGE_PATH` con permisos de escritura para el usuario del proceso.
- [ ] `NEXT_PUBLIC_API_PROXY` apunta a la URL interna de la API.
- [ ] `AI_API_KEY` presente si se quiere el modo LLM (opcional).
- [ ] Persistencia de `STORAGE_PATH` y del volumen de PostgreSQL verificada.
- [ ] La web responde y la API responde con `200` en un endpoint público.

Detalle en [`docs/operations/deployment.md`](../operations/deployment.md).

9. Patrones de diseño en el código

Referencia completa con justificación en 06-justificacion-patrones-de-diseno.md.

Patrón	Dónde	Cómo extenderlo
Singleton	`prisma/prisma.service.ts`	— (ya es global por DI)
Factory / DI con tokens	`common/guards/permission.service.ts`, `storage/`, `solicitudes/pdf/`	Registrar un nuevo provider para el token

Patrón	Dónde	Cómo extenderlo
Adapter	`storage/filesystem.adapter.ts`, `solicitudes/pdf/puppeteer-pdf-renderer.ts`, `asistente/providers/`	Implementar la interfaz y cambiar el binding del token
Proxy	`asistente/asistente.proxy.ts`	Añadir un proveedor al `resolver()`
Decorator	`common/decorators/`	Crear un decorador nuevo con `SetMetadata` y registrar el guard correspondiente
State	`convocatorias/convocatoria-state-machine.ts`, `solicitudes/solicitud-state-machine.ts`	Editar la tabla `TRANSITIONS`
Chain of Responsibility	`common/guards/permission-chain.ts`	Añadir un eslabón y una línea en `buildPermissionChain()`
Strategy	`asistente/asistente.types.ts`	Implementar `ProveedorIA` y registrar el provider

10. Extender el sistema — guías prácticas

10.1 Añadir un permiso nuevo

1. Crear el permiso en el seed (`apps/api/prisma/seed.ts`) con el formato `modulo:accion`.
2. Asignarlo al rol correspondiente en `rolPermiso`.
3. Aplicar `@Permisos('modulo:accion')` en el endpoint.
4. Documentar el endpoint en `docs/architecture/api-contracts.md`.

No se requiere desplegar para conceder o retirar un permiso ya existente: basta con modificar las filas de `rolPermiso` o `usuarioPermiso`.

10.2 Añadir un estado o una transición

1. Editar el enum en `apps/api/prisma/schema.prisma`.
2. Crear la migración: `npx prisma migrate dev --name <descripcion>`.
3. Editar la tabla `TRANSITIONS` de la máquina de estado correspondiente.
4. Actualizar `SOLICITUD_ESTADOS` / `CONVOCATORIA_ESTADOS` y, si aplica, `common/constants/estados.ts`.
5. Añadir las pruebas unitarias de la nueva transición y de las inválidas.
6. Actualizar el diagrama en `docs/architecture/diagrams.md`.

10.3 Añadir un endpoint

1. Crear el DTO con `class-validator` en `<módulo>/dto/`.
2. Añadir el método al controller con su `@Permisos(...)`.
3. Implementar la lógica en el service correspondiente.
4. Escribir las pruebas unitarias y de integración.

5. Actualizar `docs/architecture/api-contracts.md` y el README del módulo.

10.4 Migrar el almacenamiento a S3 (deuda técnica planificada)

1. Crear `S3StorageAdapter implements DocumentStorage` en `apps/api/src/storage/`.
2. Registrar el provider: `{ provide: DOCUMENT_STORAGE, useClass: S3StorageAdapter }`.
3. Añadir las variables de entorno del bucket.
4. **No tocar** `SolicitudesService` ni `SolicitudDocumentoService`.

Referencia: ADR-007 y [`docs/standards/deuda-tecnica.md`](../standards/deuda-tecnica.md).

11. Convenciones de desarrollo

11.1 Commits y ramas

```
<tipo>(<ámbito>): <descripción en imperativo y en minúsculas>

Ejemplos:
  feat(solicitudes): agregar segunda aprobación por monto
  fix(web): evitar desborde horizontal del Select en móvil
  docs(entregables): agregar justificación de patrones
  test(api): cubrir transiciones inválidas de la FSM
```

Tipos: `feat`, `fix`, `docs`, `refactor`, `test`, `chore`, `build`, `ci`.

Ramas: `feature/<modulo>-<descripcion>`, `hotfix/<descripcion>`, `fix/<descripcion>`, `docs/<descripcion>`.

11.2 Pull Requests

```
## Descripción
Qué resuelve y por qué.

## Cambios
- Archivo:línea — cambio

## Pruebas
- [ ] Unitarias
- [ ] Integración
- [ ] E2E
- [ ] Accesibilidad (si toca la UI)

## Checklist
- [ ] DoD completo
- [ ] Documentación actualizada
- [ ] Sin secretos ni ficheros de build
```

Revisión: **al menos una aprobación** de otro miembro. Merge por **squash** en `develop`.

11.3 Estilo de código

```
npm run lint      # ESLint
npm run format    # Prettier (escribe)
```

Ver [`docs/standards/code-conventions.md`](../standards/code-conventions.md).

12. Seguridad

Medida	Dónde
Contraseñas con bcrypt (coste ≥ 10)	`auth/auth.service.ts`

Medida	Dónde
Tokens access/refresh con secretos distintos	`auth/auth.service.ts`, `auth.controller.ts`
Autorización *fail-closed*	`common/guards/permissions.guard.ts` + `permission-chain.ts`
Validación estricta de DTOs (`whitelist`, `forbidNonWhitelisted`)	`main.ts` (pipes globales)
Límite de intentos de login (5 / 15 min por `ip:email`)	`auth/login-attempt.service.ts`
Validación de ficheros: MIME, extensión, ≤ 5 MB, sin path traversal	`storage/filesystem.adapter.ts` + Multer (`fileFilter`)
Auditoría y bitácora *append-only*	`audit/`, `bitacora/`
Anonimización en la eliminación de cuenta	`auth/eliminar-cuenta.service.ts`
Sin secretos versionados	`.gitignore` + escaneo en CI

Directrices completas: [`docs/standards/security-guidelines.md`](../standards/security-guidelines.md).

13. Troubleshooting

Síntoma	Causa probable	Solución
La API no arranca con error de conexión	`DATABASE_URL` incorrecto o PostgreSQL detenido	`docker compose ps`; verifica `.env`; `npm run up`
`PrismaClient is not defined`	Cliente no generado	`npx prisma generate` en `apps/api`
`P3018` al aplicar migraciones	Migración con conflicto o BOM	Revisa `apps/api/prisma/migrations/`; corrige el SQL y regenera
Puerto 3000/3001 ocupado	Otra instancia corriendo	`npm run up:stop` o libera el puerto
La web no carga datos	`NEXT_PUBLIC_API_PROXY` mal configurado o API caída	Verifica la URL en `apps/web/.env.local` y que la API responda en `/api/docs`
El PDF no se genera	Chromium no disponible	Deja `CHROME_EXECUTABLE_PATH` vacío (Puppeteer descarga el suyo) o define la ruta de un Chromium del sistema
El asistente no responde con LLM	`AI_API_KEY` no definida o inválida	Funciona en modo `fallback-kb`; revisa `AI_BASE_URL` y el timeout `AI_TIMEOUT_MS`
Error 429 en el login	Se agotaron los 5 intentos de la ventana de 15 min	Espera la ventana o reinicia el proceso (el límite es en memoria)

Síntoma	Causa probable	Solución
Las pruebas E2E fallan por datos	La base `sigeb_test` no se resiembra	El `globalSetup` la resiembra; verifica que el job tenga PostgreSQL disponible
La cobertura no alcanza el umbral	Pruebas nuevas sin cubrir	Añade pruebas o ajusta el umbral en `apps/api/jest.config`

Guía completa: [`docs/operations/troubleshooting.md`](../operations/troubleshooting.md).

14. Deuda técnica conocida

El equipo documenta honestamente lo que **no** está resuelto, con su riesgo y su mitigación, en [`docs/standards/deuda-tecnica.md`](../standards/deuda-tecnica.md). Resumen:

Deuda	Riesgo	Mitigación
Sin capa Repository	Acoplamiento del dominio a Prisma	Deuda aceptada; migración de ORM costosa
Sin bus de eventos	Acoplamiento de secuencia	Revisarlo si el dominio supera 12 módulos
`RolesGuard` y `@Roles()` sin uso	Código muerto	Pendiente de limpieza
Rate limiting en memoria	No escala ni sobrevive a un reinicio	ADR-022; la interfaz permite cambiar a Redis
`SOLICITUD_ESTADO` desalineado con la FSM	Riesgo de bug por constante divergente	Pendiente de sincronización
Sin endpoint `/health`	Healthcheck contra un endpoint de catálogo	Pendiente
Circuit breaker sin contador de fallos	Solo degradación con timeout	Pendiente de mejora en el Sprint 9
Sin caché de aplicación	Más carga sobre PostgreSQL	Aceptable con el volumen actual

15. Referencias

Documento	Contenido
[Documento de Arquitectura](04-documento-arquitectura.md)	Esqueleto del sistema y diagramas
[Justificación de la Arquitectura](05-justificacion-arquitectura.md)	Criterios del enunciado
[Justificación de Patrones](06-justificacion-patrones-de-diseno.md)	Patrones con las 4 preguntas
[Documento de Diseño](07-documento-diseno.md)	Casos de uso, componentes, clases, reglas de negocio
[Plan de Calidad](08-plan-aseguramiento-calidad.md)	Normas, procesos y métricas

Documento	Contenido
[Plan de Pruebas Ágil](09-plan-pruebas-agil.md)	Estrategia de pruebas
[`docs/architecture/ARQUITECTURA_TECNICA_DETALLADA_SIGEB_v1.1.md`](../architecture/ARQUITECTURA_TECNICA_DETALLADA_SIGEB_v1.1.md)	Monorepo, despliegue, JWT, IA en detalle
[`docs/operations/setup-guide.md`](../operations/setup-guide.md)	Guía de setup
[`docs/operations/deployment.md`](../operations/deployment.md)	Procedimientos de despliegue
[`docs/manual-usuario/`](../manual-usuario/README.md)	Manuales de usuario por rol

Última actualización: 2026-10-28